

The Humidity Controller for Mushroom Cultivation House

Student Name: Budsaya Laohapensaeng

Student ID: 6331501039

Major: Computer Engineering

Abstract— This paper presents the design of the humidity controller for a mushroom cultivation house. The controller consists of two main parts: the command node and the control node. The design of each part is explained in detail. The test result has shown that the proposed controller is operating correctly.

Keywords—Humidity Controller, Mushroom

I. INTRODUCTION

Humidity is one of the most crucial factors for growing mushrooms, as it affects their growth, development, and quality. However, maintaining the optimal humidity level for mushroom cultivation is not easy, as it depends on various factors such as the type of mushroom, the substrate, the climate, and the ventilation. Too low or too high humidity can cause problems such as drying, cracking, contamination, or poor fruiting. Therefore, a humidity controller is a device that regulates and maintains the desired humidity level for mushroom cultivation by using sensors, actuators, and feedback mechanisms. There are several types of humidity controllers available for mushroom cultivation, such as manual, semi-automatic, or fully automatic. Manual controllers require the grower to monitor and adjust the humidity level by using devices such as misting spray bottles, fans, or humidifiers. Semi-automatic controllers use sensors to measure the humidity level and turn on or off the devices accordingly, but the grower still needs to set the desired humidity level and check the system regularly. Fully automatic controllers use sensors and microcontrollers to measure and control the humidity level according to the preset values and algorithms, and they can also display and record the data for analysis and optimization. One example of a fully automatic humidity controller is the Mushroom Climate Controller, which is designed for small to medium scale mushroom cultivation [1]. This device uses a capacitive humidity sensor and a relay module to control the misting system and the fan. It has an LCD display that shows the current humidity level, the set point, and the status of the devices. It also has buttons to adjust the set point and the hysteresis. The device can be powered by a 12V DC adapter or a battery, and it has a low power consumption and a long service life. Another example of a humidity controller is the W2023 Digital Humidity Controller, which is designed for high humidity environments [2]. This device uses a specially designed sensor probe that provides accurate humidity readings to 0.1%RH. It has a control range from 5%-99%RH and can be connected to various devices such as mist makers, fans, or humidifiers. The device can be set to turn on or off the connected devices when the humidity level reaches the

desired value. This way, the device can automatically maintain the ideal humidity level for mushroom cultivation.

This paper proposes the design of humidity controller based on using the Node-red and MQTT, which allows the user to be able command and interact with the controller through IoT from anywhere and anytime.

II. THEORY

A. The proposed humidity controller

Fig. 1 shows the system diagram of the proposed humidity controller. It consists of two main parts: the command node and the control node. These nodes can communicate with each other via the MQTT protocol, as depicted in Fig. 2. The mist spray and ventilation fan are used to adjust the humidity in the mushroom cultivation house. When the value of the humidity is lower than the setpoint, the control node will command to turn on the mist spray and turn off if the value is higher., while the ventilation fan only turns on when the humidity is higher than the setpoint.

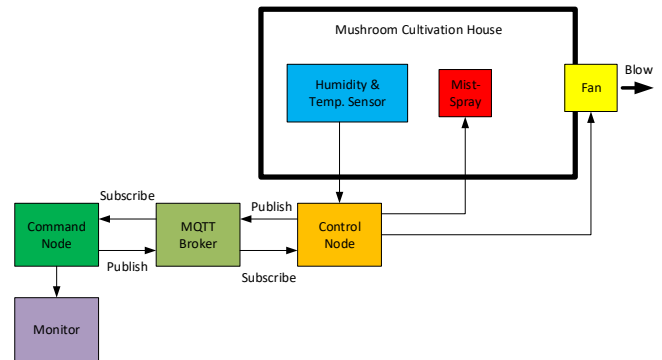


Fig1 System diagram of proposed humidity controller

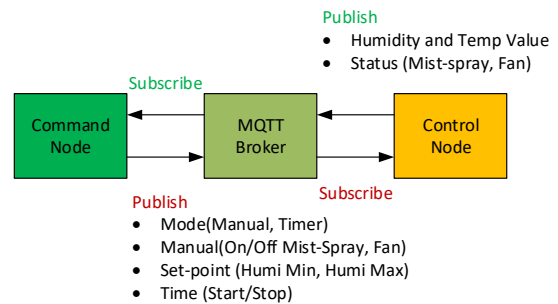


Fig2 MQTT protocol

B. Command Node

The functionality of the Command node is used for controlling the Control node to perform the desired function of controlling: manual mode and timer mode. The manual mode allows the user to command the mist spray and the ventilation fan directly, while the timer mode lets the control node turn on and off according to the setting time. It can be seen that the command mode needs a processor with high computational ability. A single-board computer is adopted as the command node in this design, as shown in Fig. 3, and Ubuntu is used as the operating system of the board.



Fig 3 The Command Node

C. Dashboard Design

Besides that, the command node can communicate, i.e., display the sensor values, to the user through the dashboard. In this design, the dashboard is created by using the Node-Red program [3]. The Node-Red is a low-code programming tool for creating and connecting devices, sensors, and services in new and interesting ways. It provides a browser-based editor that makes it easy to wire together flows using the wide range of nodes in the palette that can be deployed to its runtime in a single-click. Node-Red is built on Node.js, which makes it ideal to run at the edge of the network on low-cost hardware such as the Raspberry Pi as well as in the cloud. Node-Red can install on the local computer, a device, or a cloud platform and follow the guides on the Node-Red website. Some of the features of Node-Red are:

- Browser-based flow editing: the user can create, edit, and deploy your flows using a web browser, and use the rich text editor to write JavaScript functions.
- Built-in library: user can save and reuse useful functions, templates, or flows, and share them with the online flow library.
- Social development: thus, users can import and export the flows using JSON, and collaborate with other developers using GitHub or other platforms.
- Extensible palette: can use the wide range of nodes in the palette to connect hardware devices, APIs, and online services, and also add new nodes from the Node's package repository or create your own.

Fig.4 shows the dashboard that was created. The current temperature and the humidity from the sensors are displayed by the gauge meter in the top left. The code (flow) of reading the sensors is depicted in Fig.5. The left side of the code is used to read the data from the MQTT broker. The data is in the Json format will be extracted by the function "Data" that is written by Java script. Then, the data which is the current sensor value will be sent to display on the gauge meter in the right side of the code.

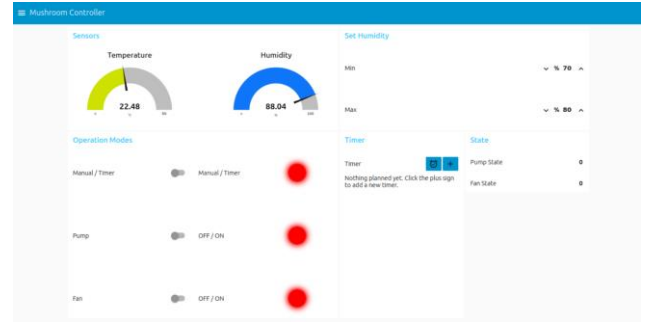
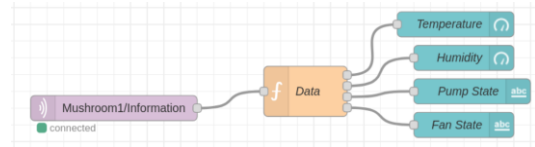


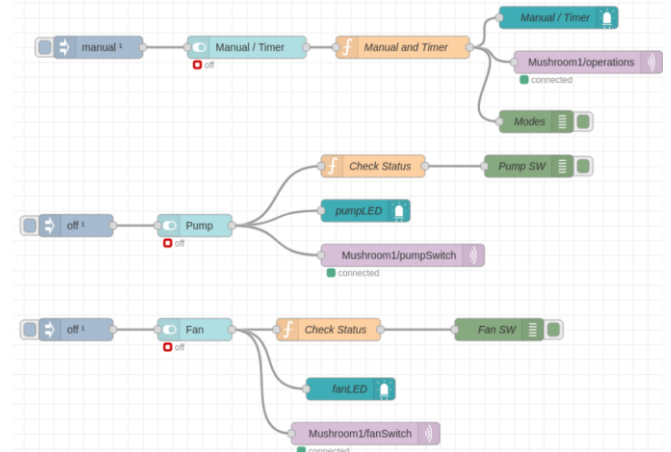
Fig 4 Dashboard



```
1 var data = msg.payload;
2 var temp = {payload:data.temp};
3 var humi = {payload:data.humi};
4 var pump = {payload:data.pump};
5 var fan = {payload:data.fan};
6 return [temp,humi,pump,fan];
```

Fig.5 Code of reading the sensors.

The position below of the gauge meter is the area of operation mode. The user must either select between manual mode or timer mode. The source code of this part is shown in Fig.6. The default mode is manual, an LED is used to indicate the status of the mode. The important part of the source code is the function "Manual and Timer". The function uses if-else condition to assign the value of variables in each mode. Then, these variables will form the control node to perform the control function according to the user desirable.



```
1 var state;
2 if(msg.payload=="timer")
3 {
4     state="timer";
5     msg.payload = state;
6     flow.set("timer",true);
7 }
8 if(msg.payload=="manual")
9 {
10    state="manual";
11    msg.payload = state;
12    flow.set("timer",false);
13 }
14 return msg;
```

Fig.6 Code of selecting the operation mode.

At the top right position is the area for assigning the setpoint and the setting time when the timer mode was selected. The desired range of humidity (Humi-min and Humi-max) are used as the setpoint value. Besides that, in this mode the user needs to set the time for start and stop the control node. The source code of this area is shown in Fig.7

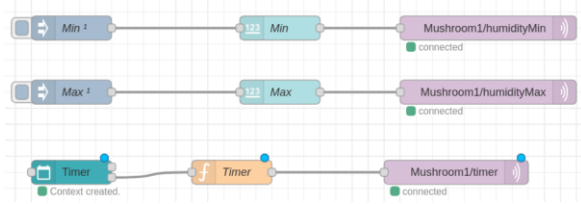


Fig.7 Code of setpoint and setting time for timer.

D. MQTT Broker

MQTT [4, 5] stands for Message Queuing Telemetry Transport, which is a lightweight and open-source protocol for machine-to-machine communication. MQTT broker is a server that handles the communication between MQTT clients, such as devices, sensors, and applications. MQTT broker has three main functions:

- Establishing connections: MQTT broker accepts and maintains the connections from MQTT clients and assigns a unique client identifier to each client. MQTT broker also supports secure connections using TLS/SSL encryption.
- Routing messages: MQTT broker uses a publish/subscribe model to route messages between MQTT clients. MQTT clients can publish messages to a topic, which is a hierarchical name that identifies the content of the message. MQTT clients can also subscribe to one or more topics and receive messages that match their subscriptions. MQTT broker ensures that the messages are delivered to the appropriate subscribers, and also handles the quality-of-service levels, which determine the reliability and delivery guarantees of the messages.
- Managing sessions: MQTT broker keeps track of the state and subscriptions of each MQTT client and stores the messages that are not yet delivered to the clients. MQTT broker also supports persistent and non-persistent sessions, which determine whether the broker retains the state and subscriptions of the clients after they disconnect. MQTT broker also supports the last will and testament feature, which allows the clients to specify a message that will be published by the broker if they unexpectedly disconnect.

As mentioned above MQTT broker is a server that handles the communication between MQTT clients, here the MQTT broker is installed on the Command node computer as the local host.

E. ESP32 Microcontroller board

The functionality of the control node is to read the temperature and the humidity from the sensor which is installed in the mushroom cultivation house. The other

functionality is to perform the control of the mist-spray and the fan. To design the control node, the ESP32 microcontroller is used. The control node reads the current sensor to its microprocessor via the I2C protocol and sends the control signal to control the mist-spray and the fan from the digital pins. Fig 9 shows the ESP32 microcontroller board for the project.

```
tearavisi@redhome:~$ sudo service mosquitto status
[sudo] password for tearavisi:
* mosquitto.service - Mosquitto MQTT Broker
   Loaded: loaded (/lib/systemd/system/mosquitto.service; enabled; preset: enabled)
   Active: active (running) since Mon 2023-12-18 19:02:26 +07; 4h 45min ago
     Docs: man:mosquitto.conf(5)
           man:mosquitto(8)
   Process: 915 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)
   Process: 920 ExecStartPre=/bin/choom mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)
   Process: 932 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)
   Process: 933 ExecStartPre=/bin/choom mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)
   Main PID: 946 (mosquitto)
     Tasks: 1 (limit: 4407)
    Memory: 272.0K
       CPU: 0.916s
   CGroup: /system.slice/mosquitto.service
           └─/usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf

Dec 18 19:02:26 redhome systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...
Dec 18 19:02:26 redhome systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.
```

Fig 8 Check status of MQTT Broker on local host



Fig 9 The ESP32 microcontroller Board

The board comes with a magnetic relay, which acts as a digital switch to turn on and off the operation of the mist spray and the fan. Also, this board provides the LEDs on board that can be used as indicators for the status of the relay.

F. Temperature and Humidity Sensor

In this project, the sensor used for measuring the temperature and the humidity within the cultivation house is AHT20 [6]. AHT20 is a new generation of temperature and humidity sensor that uses a standard I2C interface and a dual-row flat and no-lead SMD package. It has a typical accuracy of $\pm 2\%$ relative humidity and $\pm 0.3^\circ\text{C}$ at 20-80% RH and 20-60 $^\circ\text{C}$. It is suitable for reflow soldering and various applications such as industrial, consumer, weather, and IoT. Fig.10 shows the configuration and the typical connection to the microcontroller of the AHT20.

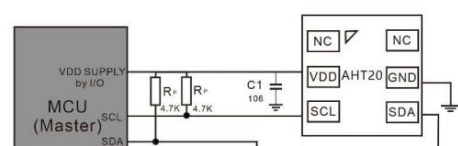


Fig.10 AHT20 sensor

G. OLED Module

To display the temperature and the humidity value read the sensor AHT20 at the control node to the user, an OLED display module is used. OLED stands for organic light-emitting diode [7,8] a type of display technology that uses organic compounds to emit light. OLED displays have high picture quality, flexibility, and lower power consumption than LCD displays. Some of the benefits of OLED displays are:

- They can produce true blacks and infinite contrast ratios, as each pixel can be individually turned on or off [8].
- They have wider viewing angles and faster response times than LCD displays, as they do not require a backlight or a color filter.
- They can be made thin, light, and flexible, as they do not require a rigid substrate or a glass cover.

Some of the drawbacks of OLED displays are:

- They are more expensive and less available than LCD displays, as they are more difficult and costly to manufacture.
- They have shorter lifespans and higher risk of burn-in than LCD displays, as the organic materials degrade over time and can retain images on the screen.
- They have lower brightness and efficiency than LCD displays, as they are affected by ambient light and heat.

An ESP32 is a microcontroller that can communicate with an OLED module using I2C or SPI protocol [9]. The user can use an OLED module with ESP32 to display text, graphics, animations, and other information. There are different types and sizes of OLED modules, such as 128x64, 128x32, white, blue, or dual color. The most common OLED module for ESP32 is the 0.96-inch SSD1306 model, which has four pins: VCC, GND, SCL, and SDA. It can connect the VCC pin to the ESP32's 3.3V output, the GND pin to ground, and the SCL and SDA pins to the ESP32's I2C pins (GPIO 22 and GPIO 21 by default). Fig 11 shows the OLED module that is used with the control node (ESP32).

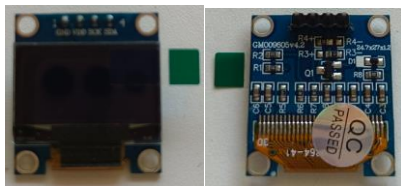


Fig 11 OLED Module

H. Switching Power Supply

A switching power supply is a type of power supply that converts AC input voltage to DC output voltage by using a switching element, such as a transistor, to control the current flow and a transformer to step up or down the voltage. A switching power supply can achieve high efficiency, low noise, and small size compared to a linear power supply. This project, the switching power supply model S-10-5 [10] as

shown in Fig.12 is adopted. S-10-5 is a model of a switching power supply that can provide 5V DC output voltage at up to 2A current from an AC input voltage range of 85-264V. It has a metal case, a terminal block connector, and an LED indicator. It is suitable for various applications, such as industrial control, LED lighting, and communication equipment



Fig 12 A switching power supply model S-10-5

I. Control Node

Fig 13 shows the control node which is the integration of the devices as mentioned previously into the waterproof box.



Fig13 Control Node

J. Database

The proposed humidity controller has a storage function. It can be created by using MySQL. Fig.14 shows the table of the database that was created. It consists of 6 columns including: Record, Temp, Humi, Mode, Fan, Pump. Fig 15 shows the code written by Node-Red to insert the data to the database. Besides that, this code can select the data to show on the table as depicted in Fig 16.

One of the very useful for handling the data is to show the data graphically. To add this function to the proposed controller, Grafana has been used. Grafana is a platform for creating and sharing dashboards that combine data from multiple sources, such as Kubernetes, logs, metrics, and traces. Node-RED is a low-code programming tool for creating and connecting devices, sensors, and services in new and interesting ways. To use Grafana with Node-RED, we need to have both tools installed and running on the system or device. It also needs to have a data source that can store and provide the data for Grafana, such as InfluxDB, MongoDB, or MySQL. We can use Node-RED to collect, process, and send the data to the data source using nodes, such as MQTT, HTTP, or InfluxDB. We can then use Grafana to connect to the data source and create dashboards with various visualizations, such as graphs,

gauges, tables, or maps. Fig 17 shows the dashboard obtained from Grafana which we use MySQL as data source.



#	Name	Type	Collation
1	Record	datetime	
2	Temp	float	
3	Humi	float	
4	Modes	varchar(10)	utf8mb4_general_ci
5	Fan	varchar(10)	utf8mb4_general_ci
6	Pump	varchar(10)	utf8mb4_general_ci

Fig.14 Database

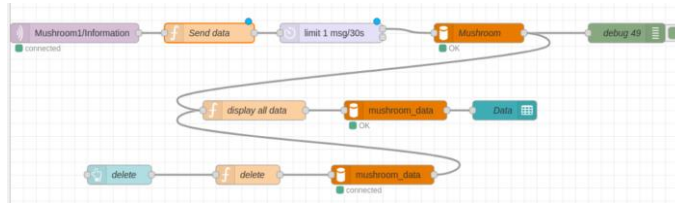


Fig 15 Code for handing the data to the database.

Date	Temp	Humidity	Mode	Fan	Pump
05:17:59 PM	26.92	68.21	manual	0	1
05:18:29 PM	26.87	65.2	manual	0	1
05:19:06 PM	26.82	67.53	manual	0	1
05:25:28 PM	26.33	69.56	timer	0	1
05:25:58 PM	26.29	69.6	timer	0	1

Fig 16 Code for selecting the data from the database to show on the table.

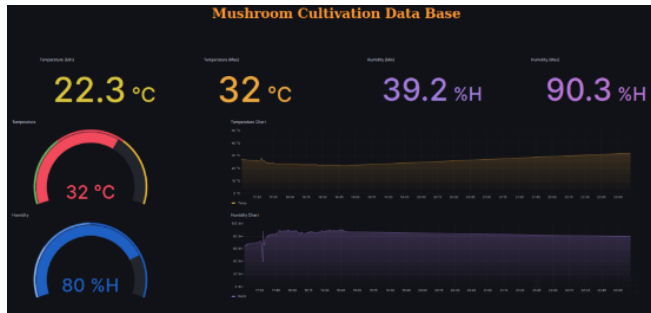


Fig 17 Dashboard from Grafana

III. EXPERIMENT

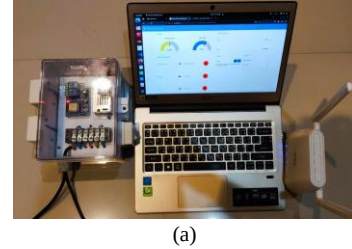
This section explains the test result of the proposed humidity controller. To test it, we begin with building a prototype: the humidity controller. Then, we set up the test system as shown in Fig. 18. Note that we use the notebook instead of the single board computer for convenience test.

A. Manual Mode

Fig 19 shows the test result of the prototype in manual mode. The result shows that it works correctly. The pump of the Mist-spray is in the off-on state according to the control position on the dashboard. Also, ones we test the ventilation fan in manual mode, it works correctly as well.



Fig.18 Set up the experiment.



(a)



(b)

Fig.19 Test result of the Mist-spray pump
Off State (LED is off), (b) On State (LED is on)

B. Timer Mode

To test the timer mode, we installed the humidity controller in the mushroom cultivation house, which was built for the experiment as shown in Fig. 20. Then the desired humidity value was set to be between 70 and 75% RH. At the beginning, we found that the humidity was not in the range; the value of the humidity was below 70% RH. Then, the controller tries to increase the humidity by turning on the mist spray. As we can see in Fig. 21, when the humidity is in the desired range, the controller will turn off the mist spray and try to keep the humidity in the desired range.



Fig.20 Install the Controller in the Mushroom cultivation House.

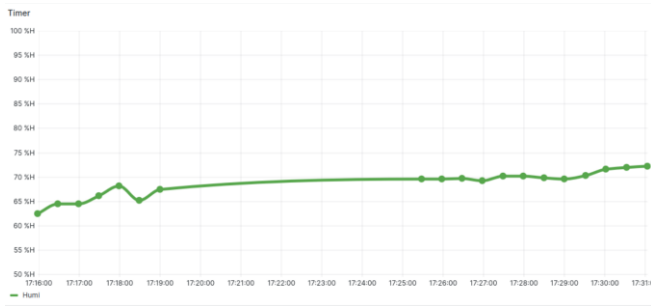


Fig.21 The test humidity versus time.

IV. CONCLUSION

The design of the humidity controller for a mushroom cultivation house is presented. The system consists of two parts: the Command Node and the Control Node. These nodes can communicate with each other via the MQTT protocol. The mist spray and the ventilation fan are used to adjust the humidity in the mushroom cultivation house. The dashboard is created by using the low-code programming tool Node-Red program. In order to test the validity of the design, a prototype was fabricated and tested. It was shown that the proposed humidity controller can operate well as expected.

REFERENCES

- [1] Ariffin, M. A. M., Ramli, M. I., Ismail, M., & Amin, M. N. M. (2020). Automatic climate control for mushroom cultivation using IoT approach. In 2020 IEEE 10th International Conference on System Engineering and Technology (ICSET) (pp. 1-6). IEEE.
- [2] Arman Kuzubasoglu, B. (2022). Recent studies on the humidity sensor: A mini review. *ACS Applied Electronic Materials*, 4(12), 4797-4807.
- [3] Node-Red: A Low-Code Programming Tool for Wiring Together Hardware Devices, APIs and Online Services. Node-Red Website. Retrieved from <https://nodered.org/> on December 19, 2023.
- [4] MQTT. (n.d.). MQTT - The standard for IoT messaging. Retrieved December 19, 2023, from <https://mqtt.org/>
- [5] OASIS. (2019). MQTT version 5.0. OASIS standard. Retrieved from <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [6] AHT20. (n.d.). AHT20 - Arduino Reference. Retrieved December 19, 2023, from <https://reference.arduino.cc/reference/en/libraries/aht20/>
- [7] OLED. (n.d.). OLED - Wikipedia. Retrieved December 19, 2023, from <https://en.wikipedia.org/wiki/OLED>
- [8] Mertens, R. (2019). *The OLED Handbook*. OLED-Info
- [9] ESP32 OLED Display with Arduino IDE. (n.d.). Retrieved December 19, 2023,
- [10] Switching Power Supply Technical Manual. (n.d.). Retrieved December 19, 2023 from <https://www.adminstrumentengineering.com.au/files/mean-well-user-technical-manual.pdf>